CartShare

A Collaborative Real-Time Shopping Cart

Project Report

Detail	Information
Student Name	[Krish Mochi]
Batch ID	[Batch10]
GitHub Repository	[krishmochi-netizen/cartshare]
Live Deployment (Netlify)	[CartShare — Collaborative Shopping Cart]
Submission Title	Batch10_KrishMochi_CartShare

1. Project Problem Statement

In shared living and working environments such as student dorms, office teams and travel groups, buying things together online is often disorganised. People currently rely on scattered tools like WhatsApp chats, sticky notes or spreadsheets to note down who needs what, and these tools were never designed for collaborative shopping. The project addresses three connected problems:

- **The communication gap.** Requests are spread across several chats and notes. Items get missed, bought twice, or have to be chased repeatedly, which wastes time and causes frustration.
- **The delivery threshold trap.** Many online platforms offer free shipping only above a minimum order value (for example $75). Without a shared view of the running total, groups find it hard to know how far they are from the threshold, so they either overspend or pay avoidable delivery fees.
- **Lack of transparency.** It is hard to see who added which item. When the order arrives, splitting the bill fairly becomes a tedious and sometimes disputed exercise.

There was a clear need for one simple shared place where a group can build a cart together, see updates immediately, track their spending and generate a clear record of who owes what.

2. Project Solution

CartShare is a responsive web application that turns a static frontend prototype into a collaborative shopping tool. A user enters a name and a room code, and everyone who joins the same room works on one shared cart that updates live across browser tabs.

2.1 How the problem is solved

- **Rooms with a unique code:** A user can create a room (a random 6-character code is generated) or join an existing one. This gives every group a private shared space.
- **Shared cart:** Participants add, edit and remove items. Each item records who added it and when, so nothing is anonymous.
- **Free-delivery tracker:** A progress bar shows the cart total against the delivery threshold and tells the group how much more is needed.
- **Real-time activity log:** Every action (joining, adding, removing, changing quantity) is logged with the user name and time, creating a clear history.
- **Fair expense split:** The cart shows a per-person subtotal, so the bill can be divided without confusion.
- **Printable receipt:** A clean, audit-ready receipt can be generated and printed or saved as PDF.

2.2 Tools and methods used

Requirement	Technology	How it was used
User access	HTML forms + JavaScript	Name and room-code form with validation; session stored per tab so each tab can act as a different user.
Responsive UI	CSS Flexbox, Grid, Bootstrap 5	Mobile-first layout; cart, form and log stack on phones and sit side by side on desktop.
Data persistence	Browser localStorage	Room data, cart items and activity log are stored so they survive page refreshes.
Collaboration	JavaScript storage event / BroadcastChannel	Changes made in one tab are detected by other tabs, which re-render instantly.
Printing	CSS @media print + window.print()	Dedicated print stylesheet hides buttons and navigation and formats the receipt for A4.
Hosting	GitHub + Netlify	Source code on GitHub; static site deployed live on Netlify.

The code is organised in modules (storage, authentication, room, sync and receipt), with all localStorage access kept in one file. User input is escaped before display to prevent script injection, and cart updates are merged by item ID so that two tabs editing at the same time do not overwrite each other.

3. Project Report

3.1 Project overview

CartShare is a front-end only project built with HTML5, CSS3, JavaScript (ES6) and Bootstrap 5. It runs entirely in the browser and is deployed as a static website, so it needs no server or database. The application has three pages: a login and join page, the shared room with the cart and activity log, and a printable receipt page.

3.2 Folder structure

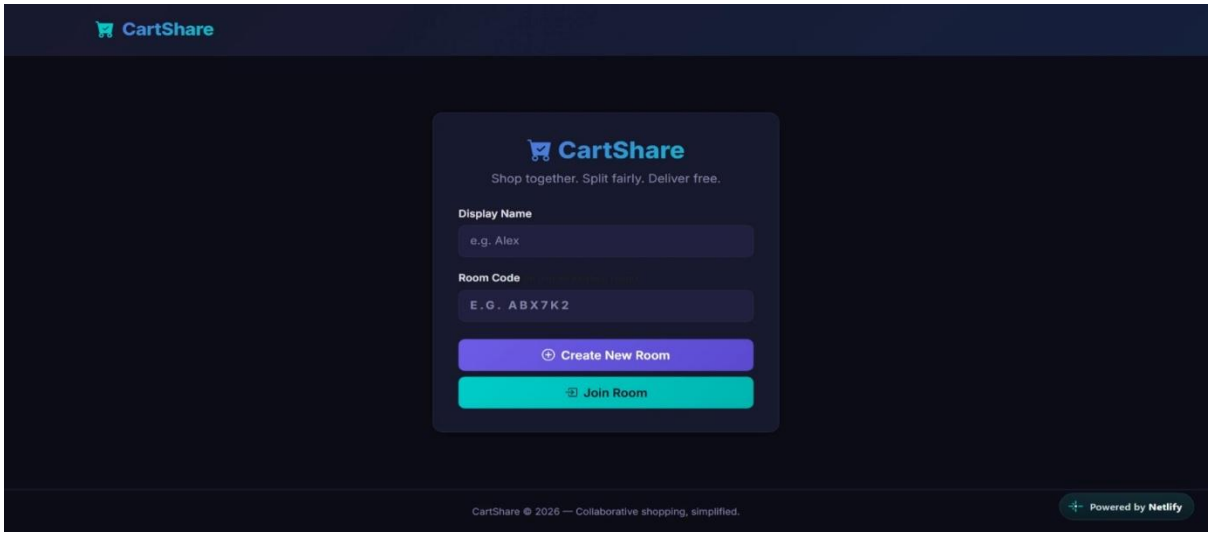
Path	Purpose
index.html	Login page: create or join a room
room.html	Shared cart, participants and activity log
receipt.html	Printable group receipt
css/	style.css (main styles) and print.css (print rules)
js/	storage.js, auth.js, room.js, sync.js, receipt.js
assets/	Images and icons
README.md	Run instructions and feature documentation

3.3 How to test multiple users

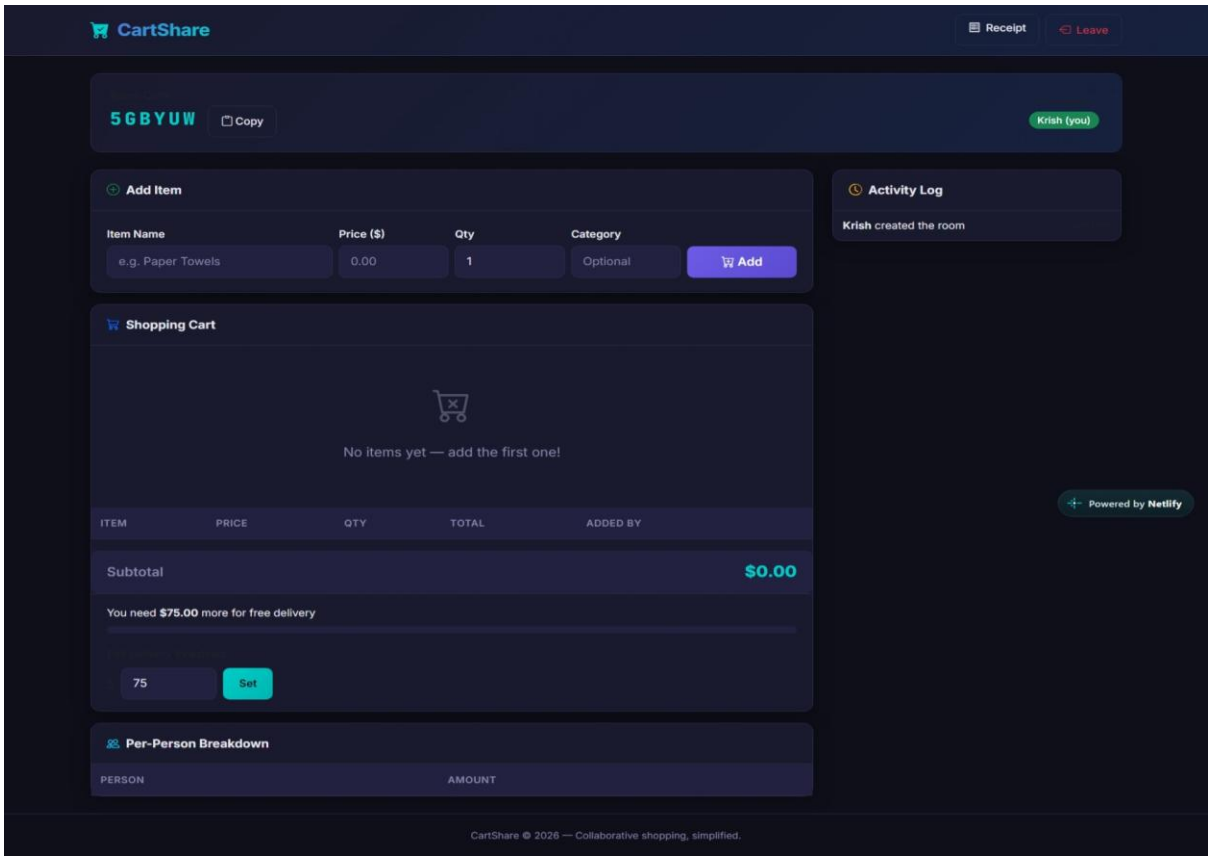
Open the live link in two or three tabs of the same browser. In the first tab create a room and copy its code. In the other tabs join with the code, using a different name in each. Add and remove items in any tab and observe that the cart, totals, progress bar and activity log update in all of them without refreshing.

3.4 Screenshots

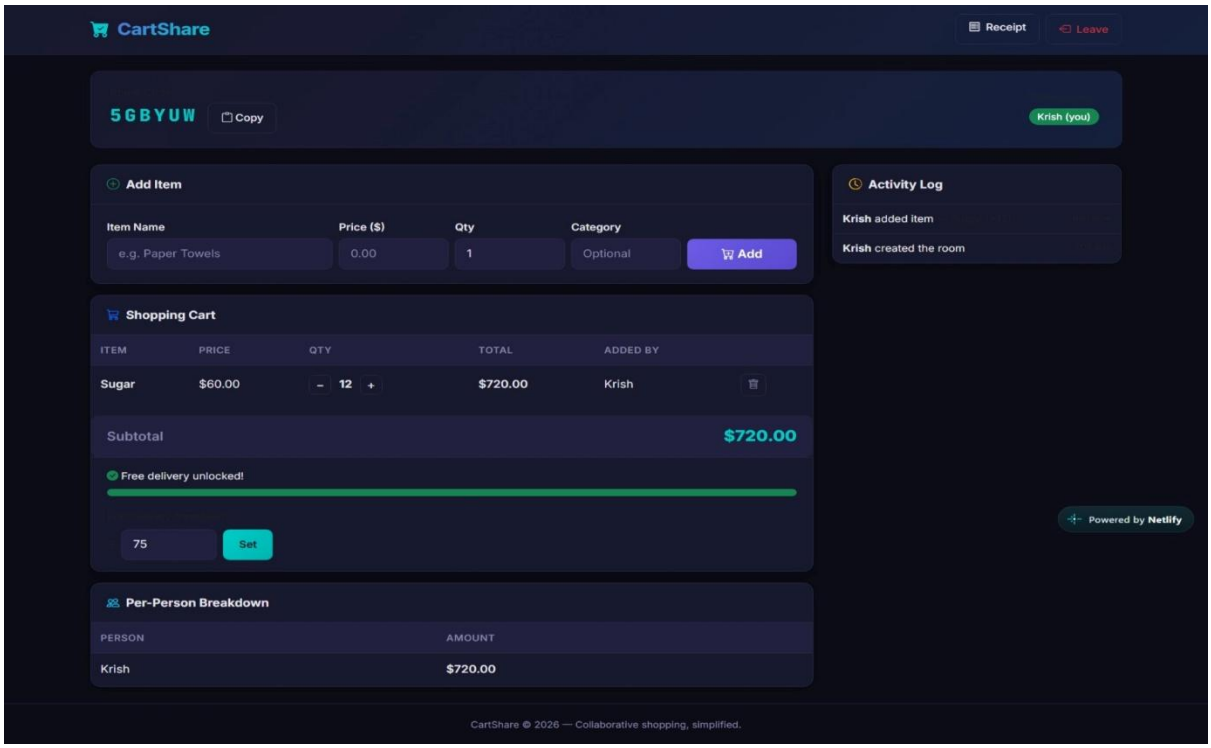
1.



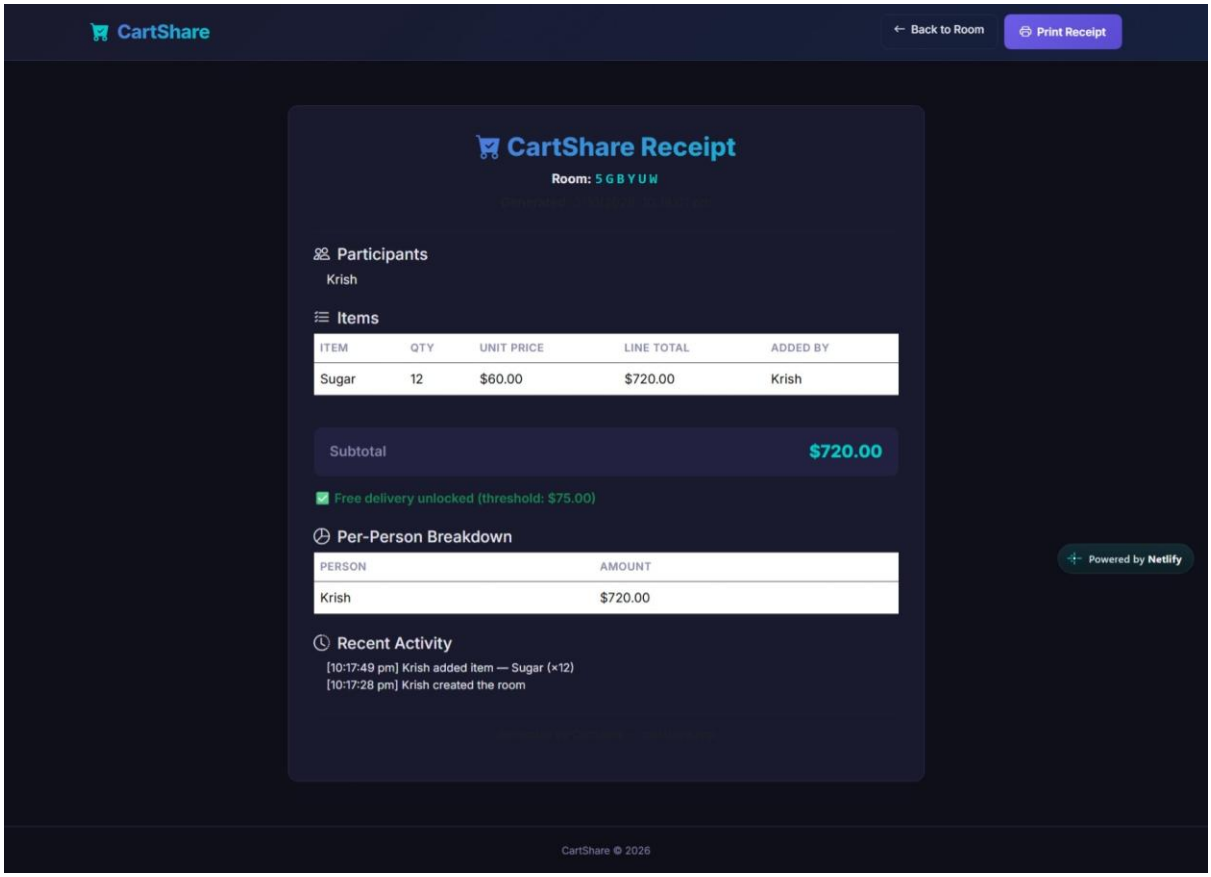
2.


CartShare
Receipt
Leave
5GBYUW
Copy
Krish (you)
Add Item
Item Name
e.g. Paper Towels
Price ($)
0.00
Qty
1
Category
Optional
Add
Activity Log
Krish created the room
Shopping Cart
No items yet — add the first one!
ITEM
PRICE
QTY
TOTAL
ADDED BY
Subtotal
$0.00
You need $75.00 more for free delivery
75
Set
Per-Person Breakdown
PERSON
AMOUNT
Powered by Netlify
CartShare © 2026 — Collaborative shopping, simplified.

3.


CartShare
Receipt
Leave
5GBYUW
Copy
Krish (you)
Add Item
Item Name
e.g. Paper Towels
Price ($)
0.00
Qty
1
Category
Optional
Add
Activity Log
Krish added item
Krish created the room
Shopping Cart
ITEM
PRICE
QTY
TOTAL
ADDED BY
Sugar
$60.00
12
$720.00
Krish
Subtotal
$720.00
Free delivery unlocked!
75
Set
Per-Person Breakdown
PERSON
AMOUNT
Krish
$720.00
Powered by Netlify
CartShare © 2026 — Collaborative shopping, simplified.

4.



3.5 Limitations

Because the project has no backend, synchronisation works across tabs and windows of the same browser on one device. Syncing between different devices would require a real-time server such as Firebase or WebSockets, which is a natural next step.

3.6 Conclusion

CartShare gives groups a single, transparent place to plan joint purchases. It removes duplicate buying, makes the delivery threshold visible, records who added what and produces a clear receipt for splitting costs. It also demonstrates the use of forms, responsive layout, browser storage, event-driven synchronisation and print styling in one working, deployed application.